

ACQUISITIONOS · PAYMENT CONFIGURATION

Razorpay Setup Guide

Complete step-by-step walkthrough for Razorpay in AcquisitionOS — activation, API keys, one-time orders vs recurring subscriptions, GST server-side math, signature verification, webhooks, and Test → Live migration.

Orders & Subscriptions

UPI · Cards · NetBanking

HMAC verification

Webhooks

INR + 18% GST

Version 1.0 · September 23, 2026 · Documentation only — verified against the current codebase

This guide documents exactly what the **current** AcquisitionOS codebase expects from Razorpay. Everything here was verified against the source (`src/lib/payments/razorpay-provider.ts` , `src/lib/razorpay-service.ts` , `src/lib/payment-service.ts` , `src/app/api/payments/webhook/razorpay/route.ts` , `src/app/api/payments/razorpay/verify/route.ts` , `src/lib/gst-service.ts`). Only Razorpay features actually supported by the implementation are described — nothing is invented. Where the Razorpay Dashboard UI varies, instructions are marked **UI may vary by provider dashboard version** .

Part 1 — How AcquisitionOS Uses Razorpay (Read This First)

Razorpay is the **India-first gateway** (INR + 18% GST, UPI/cards/netbanking/wallets). Two distinct checkout modes exist in the code:

Mode	When it is used	Razorpay object	Renewals
One-time order (default)	Always, unless a Plan ID env var is configured for the plan/cycle	<code>razorpay.orders.create</code> with a server-computed amount (base + 18% GST, in paise)	Handled in-app by the end-of-period cron (<code>/api/cron/renew-subscriptions</code>) — no Razorpay auto-renewal
Recurring subscription	Only when the plan/cycle has <code>RAZORPAY_PLAN_<PLAN>_<CYCLE></code> set to a <code>plan_...</code> ID	<code>razorpay.subscriptions.create(plan_id, total_count, quantity, notes)</code>	Auto-charged by Razorpay; app syncs via <code>subscription.charged</code> / <code>subscription.halted</code> / <code>subscription.cancelled</code> / <code>subscription.completed</code> webhooks

Flow (one-time order):

1. User picks a plan (Pro/Elite) or a credit pack in the app.
2. App calls `POST /api/payments/create-checkout-session` (gateway `razorpay`) or `POST /api/payments/create-order` → server validates plan change, computes **base amount + GST server-side**, calls `razorpay.orders.create` , stores a `PaymentOrder` row.
3. The browser opens Razorpay Checkout.js with the public Key ID, `order_id` , amount, currency, and user prefill — the browser never computes or owns the amount.
4. On success the handler posts `razorpay_order_id` , `razorpay_payment_id` , `razorpay_signature` to `POST /api/payments/razorpay/verify` .
5. The server verifies the HMAC-SHA256 signature, **fetches the payment from Razorpay's API**, checks the amount against the stored order (± ₹0.01), then activates atomically (webhook dedup table + pending-order check make this idempotent).
6. The webhook (`/api/payments/webhook/razorpay`) is the parallel source of truth and completes activation even if the browser dies.

Key facts that drive the setup steps:

Fact	Consequence
Amounts computed server-side from the app's plan table + GST	You do not create Razorpay Plans/amounts for one-time checkouts; keep dashboard Plans (if any) at the GST-inclusive totals below so recurring charges match
Signature verification mandatory outside development	<code>RAZORPAY_WEBHOOK_SECRET</code> is required in production; unsigned webhooks are rejected with 500
Browser-reported success is never trusted	The verify route + gateway fetch + webhook are the activation paths
Starter is Stripe-only today	The Razorpay provider currently validates <code>plan</code> against Pro/Elite only ("Only Pro and Elite plans require payment"). Starter via Razorpay is rejected by design in the current code — document/configure Pro and Elite (and credit packs) here

Part 2 — Create / Log In to Your Razorpay Account

1. Go to <https://dashboard.razorpay.com/signup> and register with your business email.
2. Verify email + mobile; enable 2FA (recommended).

Part 3 — Complete Account/Business Activation

Real payments require KYC activation (Test Mode works immediately):

1. Dashboard → **Account & Settings** → **Profile** (UI may vary) → complete business profile: legal entity type, business name, PAN, GSTIN (if registered), registered address, bank account.
2. Submit website/app details (the AcquisitionOS URL + product description) for approval.
3. Wait for activation status to become **Active** under Account & Settings. Until then, Live Mode keys cannot be generated.

Part 4 — Test Mode vs Live Mode

Toggled in the Dashboard (**upper-right mode switch**, UI may vary):

Mode	Key ID prefix	Payments	Use
Test	rzp_test_..	Simulated (Razorpay's test instruments: test UPI VPA <code>success@razorpay</code> , test cards, test netbanking)	All setup + rehearsal
Live	rzp_live_..	Real money (post-KYC)	Production

The app **auto-detects mode from the key prefix** (`rzp_test_` → `mode: 'test'`) and reports it through `GET /api/payments/provider-status` . No mode setting exists in the app.

Part 5 — API Key ID and Secret

1. Dashboard → **Account & Settings** → **API Keys** → **Generate Test Key** (repeat for Live when ready) (UI may vary).
2. Copy:
3. **Key ID** → env var `RAZORPAY_KEY_ID` (used by server SDK calls and, when `NEXT_PUBLIC_RAZORPAY_KEY_ID` is absent, also sent to the browser as the Checkout.js public key — the Key ID is public-safe by design)
4. **Key Secret** → env var `RAZORPAY_KEY_SECRET` (used for HMAC signature verification, refund API calls, and gateway payment fetches — **server-only, never expose**)
5. Where they are consumed (verified): `razorpay-provider.ts` (`getRazorpayInstance` , `isConfigured`), `payment-service.ts` (`getRazorpayInstance` , availability checks), `razorpay-service.ts` (`verifyPaymentSignature` , `verifyWebhookSignature` , refunds), `refund-service.ts` .
6. Optional client-dedicated publishable copy: `NEXT_PUBLIC_RAZORPAY_KEY_ID` — preferred by the code (`getPublicKeyId()`) when present. Set it to the same `rzp_...` Key ID.

Merchant ID: Razorpay exposes account IDs in some dashboard views, but the current AcquisitionOS integration **does not use a Merchant ID anywhere** — nothing to configure.

Part 6 — Do You Need Razorpay Plans? (Monthly + Yearly)

- If you are satisfied with one-time orders + in-app renewal cron (the default): create nothing. No Plans, no dashboard configuration — amounts are computed per-order server-side.
- If you want Razorpay-native auto-renewal for Pro/Elite: create **Plans** (recurring billing objects) and map them via env vars. The app checks `getRazorpayPlanId(plan, cycle)` ; when it returns a `plan_...` ID, checkout switches to `razorpay.subscriptions.create` .

UI may vary by provider dashboard version — conceptual navigation: **Dashboard → Subscriptions → Plans → Create Plan.**

Plan matrix to create (recurring, auto-charged)

Plan · Cycle	Plan name (recommended)	Amount to enter	Interval	Notes
Pro · Monthly	Acquisition0S Pro Monthly	₹1,887 (188700 paise)	1 month	GST-inclusive total (base ₹1,599 + ₹288 GST)
Pro · Yearly	Acquisition0S Pro Yearly	₹17,699 (1769900 paise)	1 year	base ₹14,999 + ₹2,700 GST
Elite · Monthly	Acquisition0S Elite Monthly	₹6,135 (613500 paise)	1 month	base ₹5,199 + ₹936 GST
Elite · Yearly	Acquisition0S Elite Yearly	₹53,099 (5309900 paise)	1 year	base ₹44,999 + ₹8,100 GST

- Starter has no Razorpay path in the current code (provider-level plan validation) — do not create a Starter Plan; sell Starter via Stripe.
- Amounts must equal the GST-inclusive totals because `createRazorpaySubscriptionOrder` computes the same total (`getPlanPrice(base) → calculateGST`) and the verify route compares the charged amount to the stored order amount with a tolerance of ₹0.01. A Plan priced differently guarantees `AMOUNT_MISMATCH` rejections on renewals.

Copy each Plan ID into the exact env var

Plan · Cycle	Env var (exact)	Value format
Pro · Monthly	RAZORPAY_PLAN_PRO_MONTHLY	plan_...
Pro · Yearly	RAZORPAY_PLAN_PRO_YEARLY	plan_...
Elite · Monthly	RAZORPAY_PLAN_ELITE_MONTHLY	plan_...
Elite · Yearly	RAZORPAY_PLAN_ELITE_YEARLY	plan_...

The variable is recognized only when the value **starts with** `plan_` (verified in `plan-config.ts`). Missing/empty → one-time order fallback (still fully functional). For recurring subscriptions you may also tune mandate tenure:

Env var	Default	Meaning
RAZORPAY_SUBSCRIPTION_TOTAL_COUNT_MONTHLY	12	Number of billing cycles for monthly subscriptions
RAZORPAY_SUBSCRIPTION_TOTAL_COUNT_YEARLY	5	Number of billing cycles for yearly subscriptions

Part 7 — Credit Add-On Purchases (One-Time)

Credit packs work on Razorpay exactly like one-time plan orders — **no Razorpay objects needed:**

Pack	Base (₹)	GST 18% (₹)	Total charged (₹)
100 credits	499	90	589
500 credits	1,999	360	2,359

1,000 credits	3,499	630	4,129
---------------	-------	-----	-------

The order is created via `razorpay.orders.create` with `notes: { userId, plan: 'credit_addon', billingCycle: 'one_time', credits, type: 'credits' }`, then verified/fulfilled by the same signature-verify path (`fulfillCreditAddon` → credits added atomically + ledger + notifications).

Part 8 — Order IDs, Payment IDs, and Invoice IDs

How each Razorpay identifier is used by the app (no configuration required — for your reference):

Identifier	Example	Where it appears in AcquisitionOS
Order ID	<code>order_..</code>	Created per checkout (<code>razorpay.orders.create</code>), stored as <code>PaymentOrder.providerOrderId</code> , used by Checkout.js (<code>order_id</code> field) and webhook order matching
Payment ID	<code>pay_...</code>	Returned by Checkout.js and webhooks; stored as <code>PaymentOrder.providerPaymentId</code> ; fetched server-side during verification; used in refund calls
Subscription ID	<code>sub_...</code>	(Recurring mode only) stored as <code>PaymentOrder.providerSubscriptionId</code> and on the app Subscription record; used for signature fallback + lifecycle webhooks
Invoice ID	<code>inv_...</code>	Not used — AcquisitionOS generates its own numbered GST invoices (PDF + email) in-app; Razorpay invoicing stays off

Part 9 — Subscription Configuration Summary

When a Plan ID is configured, `razorpay.subscriptions.create` is called with: `plan_id`, `total_count` (env-tunable, defaults 12/5), `quantity: 1`, `customer_notify: 1`, and `notes: { paymentOrderId, userId, plan, billingCycle, couponCode? }` so the first charge's webhook can be attributed to the internal order. Nothing further to configure in Razorpay; just keep the webhook (Part 10) active so lifecycle events sync.

Part 10 — Configure the Webhook

1. **Endpoint URL (exact):** `https://<your-domain>/api/payments/webhook/razorpay`
2. Current deployment: `https://acquisition.space-z.ai/api/payments/webhook/razorpay`
3. Navigation: Dashboard → **Account & Settings** → **Webhooks** → **Add New Webhook** (UI may vary).
4. Enter the URL, choose **Active**, and select the events in Part 11.
5. Set a **webhook secret** — a strong random string you invent here. Put the **same value** in the app env var `RAZORPAY_WEBHOOK_SECRET`. (Unlike Stripe, Razorpay does not generate the secret; you define it on both sides.)
6. Save. Use the **Send test** / ping option if available.

Verification behavior (verified in code): the route computes HMAC-SHA256 of the **raw request body** with `RAZORPAY_WEBHOOK_SECRET` and compares it to the `x-razorpay-signature` header using a constant-time comparison. Missing/invalid signatures → 400. **No secret configured outside development → 500 rejection** (hard security rule). In local development only, unsigned webhooks are tolerated with a warning.

Part 11 — Required Webhook Events (Exact List)

Event	What the app does
<code>payment.captured</code>	Primary activation for one-time orders (and first charge of subscriptions): order → completed, subscription activated, credits granted, invoice generated
<code>payment.failed</code>	Marks the pending order failed, logs the reason, surfaces failure UI

<code>refund.created</code>	Refund initiated: logs and begins credit-reversal handling
<code>refund.processed</code>	Refund completed: reverses credits; downgrades plan on full refund
<code>subscription.cancelled</code>	Ends the subscription in-app (canceled/expired)
<code>subscription.charged</code>	Renewal charge: extends period, resets monthly credits (period-idempotent), generates invoice
<code>subscription.halted</code>	Recurring payment failed / mandate paused → subscription marked past-due/halted
<code>subscription.completed</code>	Subscription finished its full tenure → handled per status mapping

(Other events are recorded in the `PaymentWebhook` table and safely ignored.)

Part 12 — Test Webhook Delivery

1. With the app running locally, expose it (e.g. `ngrok http 3000`) and register the tunnel URL + `/api/payments/webhook/razorpay` as a Test-mode webhook.
2. Trigger a real Test checkout (test UPI VPA `success@razorpay` or Razorpay's test cards) — `payment.captured` arrives at your endpoint.
3. CLI-less simulation: Dashboard → Webhooks → your endpoint → **Send test event** (availability varies by dashboard version).
4. Verify app-side: the `PaymentWebhook` table gains a row (`provider='razorpay'` , `eventType` , `processed=true`) and `GET /api/admin/billing/webhooks` lists it.

Part 13 — Payment Verification (How the App Verifies)

The verify route (`POST /api/payments/razorpay/verify`) performs, in order (all verified in code):

1. **Order lookup + ownership** — internal `PaymentOrder` by ID, must belong to the authenticated user and be `pending` (completed orders return `alreadyProcessed`).
2. **Signature check** — `expected = HMAC_SHA256( RAZORPAY_KEY_SECRET , <order_id|payment_id> )` (falls back to `<subscription_id|payment_id>` for subscription checkouts), constant-time compared.
3. **Gateway fetch** — `payments.fetch(payment_id)` ; status must be `captured` or `authorized` , and the payment must reference **our** order/subscription ID.
4. **Amount check** — `gateway amount ÷ 100` vs stored order amount, tolerance ₹0.01; mismatches mark the order failed (`AMOUNT_MISMATCH`).
5. **Idempotent activation** — dedup row `checkout_verify_<paymentId>` in `PaymentWebhook` , then atomic activation (`confirmPaymentAndActivate`) or credit-add-on fulfillment.

Failure codes you may see in responses/logs: `ORDER_NOT_FOUND` , `ORDER_NOT_PENDING` , `SIGNATURE_INVALID` , `PAYMENT_FAILED` , `AMOUNT_MISMATCH` , `NOT_CONFIGURED` , `GATEWAY_ERROR` .

Part 14 — Failed, Pending, and Refund Handling

State	Behavior (verified)
Failed payment	Checkout.js handler error or <code>payment.failed</code> webhook → order marked <code>failed</code> , failure reason logged, user sees the payment-failed modal and can retry (<code>POST /api/payments/retry</code>)
Pending / processing	Order remains <code>pending</code> ; either the browser verify or the webhook completes it later; <code>/api/cron/payment-reconciliation</code> reconciles stale pendings with the gateway; a "confirming payment" state is shown meanwhile
Refunds	Dashboard refund (Payments → payment → Refund) or super-admin <code>POST /api/admin/refund</code> (calls Razorpay's refund API with Key ID/Secret). <code>refund.created</code> / <code>refund.processed</code> webhooks reverse credits via <code>refundCredits</code> and downgrade the plan on full refunds. Partial refunds supported. Only <code>completed</code> orders can be refunded

Subscription lifecycle	<code>subscription.charged</code> renewals reset credits (idempotent per period); <code>subscription.halted</code> (payment failing) → past-due handling + recovery banner; <code>subscription.cancelled</code> → access ends per status mapping
-------------------------------	--

Part 15 — Exact Environment Variables (Razorpay)

Variable	Provider	Purpose	Mode	Required?	Where to obtain
<code>RAZORPAY_KEY_ID</code>	Razorpay	Server SDK auth (orders, subscriptions, payments fetch, refunds)	test <code>rzp_test_...</code> / live <code>rzp_live_...</code>	Required for any Razorpay checkout	Dashboard → Account & Settings → API Keys
<code>RAZORPAY_KEY_SECRET</code>	Razorpay	HMAC payment verification + webhook-adjacent crypto + refund API	secret	Required with key ID	Same page (shown once — store safely)
<code>NEXT_PUBLIC_RAZORPAY_KEY_ID</code>	Razorpay	Public Key ID handed to Checkout.js in the browser	public-safe	Optional (falls back to <code>RAZORPAY_KEY_ID</code>)	Same as Key ID
<code>RAZORPAY_WEBHOOK_SECRET</code>	Razorpay	HMAC-SHA256 verification of webhook bodies	secret	Required in production (unsigned webhooks rejected)	Your own value, set identically in Dashboard → Webhooks
<code>RAZORPAY_PLAN_PRO_MONTHLY</code>	Razorpay	Recurring Pro monthly Plan ID	<code>plan_...</code>	Optional (one-time fallback without it)	Dashboard → Subscriptions → Plans
<code>RAZORPAY_PLAN_PRO_YEARLY</code>	Razorpay	Recurring Pro yearly Plan ID	<code>plan_...</code>	Optional	Same
<code>RAZORPAY_PLAN_ELITE_MONTHLY</code>	Razorpay	Recurring Elite monthly Plan ID	<code>plan_...</code>	Optional	Same
<code>RAZORPAY_PLAN_ELITE_YEARLY</code>	Razorpay	Recurring Elite yearly Plan ID	<code>plan_...</code>	Optional	Same
<code>RAZORPAY_SUBSCRIPTION_TOTAL_COUNT_MONTHLY</code>	Razorpay	Mandate tenure, monthly	config	Optional (default 12)	Your choice
<code>RAZORPAY_SUBSCRIPTION_TOTAL_COUNT_YEARLY</code>	Razorpay	Mandate tenure, yearly	config	Optional (default 5)	Your choice
<code>RAZORPAY_SUPPORTED_CURRENCIES</code>	Razorpay	Advertised currency list (default <code>INR</code>)	config	Optional	Set <code>INR, USD</code> only if your account supports international

Part 16 — Test Mode → Live Mode Checklist

- [] KYC activation complete (Account & Settings shows Active)
- [] Switch Dashboard to Live mode; generate **Live** API keys
- [] Replace `RAZORPAY_KEY_ID` / `RAZORPAY_KEY_SECRET` (and `NEXT_PUBLIC_RAZORPAY_KEY_ID`) with Live values
- [] Re-create the four Plans in **Live mode**; update the four `RAZORPAY_PLAN_*` env vars with Live `plan_...` IDs
- [] Create the **Live** webhook (same URL, same events); set `RAZORPAY_WEBHOOK_SECRET` to the Live webhook's secret
- [] `GET /api/payments/provider-status` shows `razorpay.mode = 'live'`
- [] One real live ₹1 transaction end-to-end (refund afterwards if self-test): verify → activate → invoice → webhook processed
- [] Confirm refund path works in Live (small refund on the self-test)

Part 17 — Production Verification Checklist

- [] Webhook URL publicly reachable over HTTPS; test event returns 200
- [] `payment.captured`, `subscription.charged` (if recurring), `refund.processed` observed at least once
- [] Signature tamper test: a webhook with a wrong signature is rejected with 400 (log shows "Invalid signature — rejecting")
- [] Amount-mismatch test (optional, staging): order amount modified → `AMOUNT_MISMATCH`, order failed, no activation
- [] Invoice PDF + email generated on a real activation; visible under the user's billing history (`GET /api/payments/history`)
- [] Cancel path: subscription cancellation syncs via `subscription.cancelled`
- [] Credit pack purchase adds exactly 100/500/1000 credits with a `CreditsLedger` entry

Part 18 — Troubleshooting

Symptom	Cause	Fix
Razorpay is not configured. Set <code>RAZORPAY_KEY_ID</code> and <code>RAZORPAY_KEY_SECRET</code> .	Missing env vars	Set both, redeploy; confirm with <code>provider-status</code>
Webhook 500 Webhook verification not configured	<code>RAZORPAY_WEBHOOK_SECRET</code> unset outside development	Set the same secret in Dashboard webhook + env
Webhook 400 Invalid signature	Secret mismatch, or body was re-serialized (signature covers raw bytes)	Ensure the exact secret string; proxy must forward the raw body
<code>SIGNATURE_INVALID</code> on checkout verify	Key Secret mismatch between environments, or ids from different orders	Confirm env secret matches the account that issued the keys; retry checkout
<code>AMOUNT_MISMATCH</code>	Plan priced differently from the app's GST-inclusive total, or Price/Plan edited after orders exist	Align Plan amounts with Part 6 table; never edit amounts of in-flight plans
Starter plan cannot be bought via Razorpay	Provider-level validation (Pro/Elite only)	Sell Starter via Stripe (documented behavior of current code)
Buttons show Coming Soon	Razorpay keys not configured (availability requires Key ID and Secret)	Set both env vars
Renewal charge not reflected	<code>subscription.charged</code> webhook missing/failed	Check deliveries; replay safely (PaymentWebhook dedup makes replays idempotent)
UPI fails in Test	Test UPI needs Razorpay test instruments	Use <code>success@razorpay</code> VPA or test cards from Razorpay docs

End of Razorpay Setup Guide. Companion documents: `STRIPE-SETUP-GUIDE.md` and `PAYMENT-CONFIGURATION-MASTER.md`.

Configuration complete.

Every environment variable name, endpoint, webhook event, and amount in this guide was verified against the current AcquisitionOS source code. Configure objects in Test Mode first, verify the full checklist, then repeat in Live Mode.

Companion guides: [Stripe Setup Guide](#) · [Payment Configuration Master](#)